Technical Report: AI-Assisted Resume Analyzer & ATS Optimizer

Coursework Submission
Student Name: Priyanshu Mangla
Roll Number: 2301010665 **Project Category:** Section D – Mini Project using AI Tools

1. Project Abstract

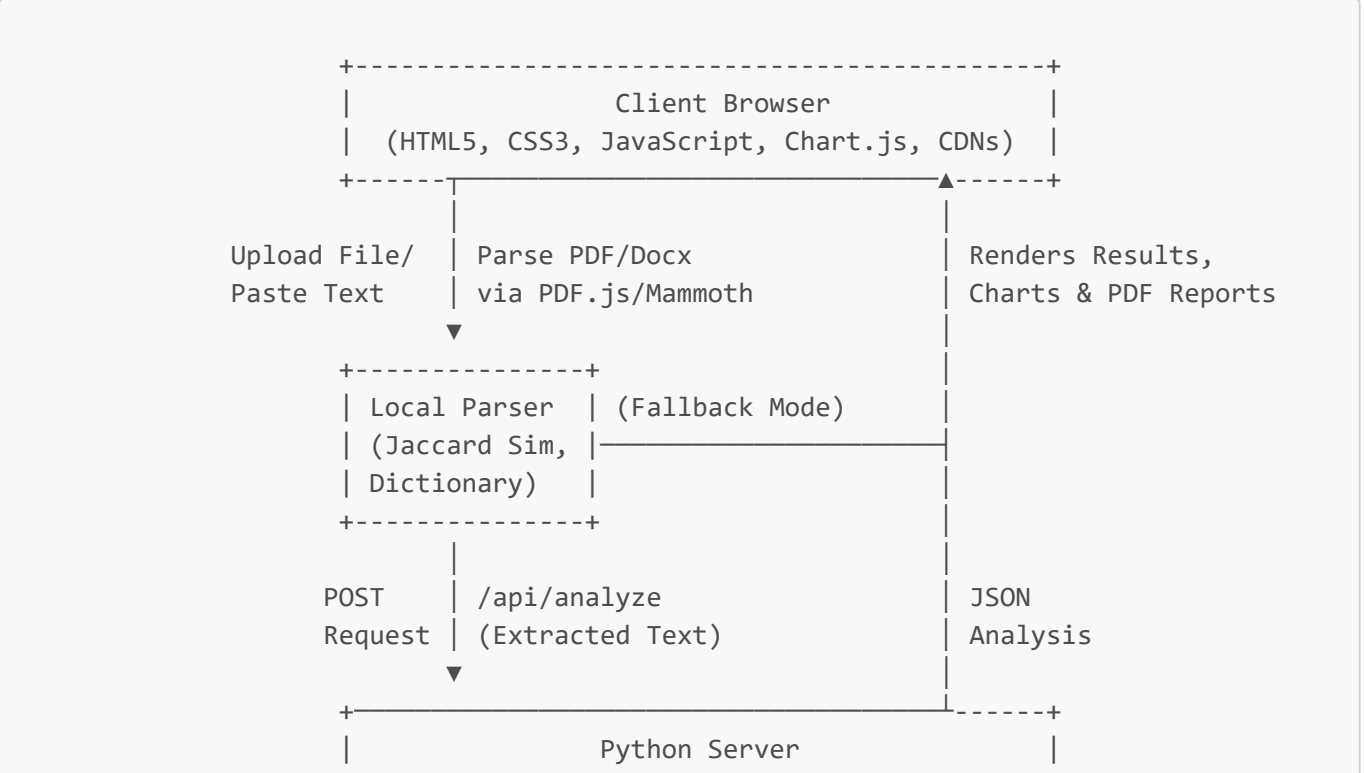
In today's highly competitive job market, recruiters process hundreds of applications using Applicant Tracking Systems (ATS). A large percentage of qualified resumes are filtered out due to structural issues, keyword mismatches, or missing domain skills.

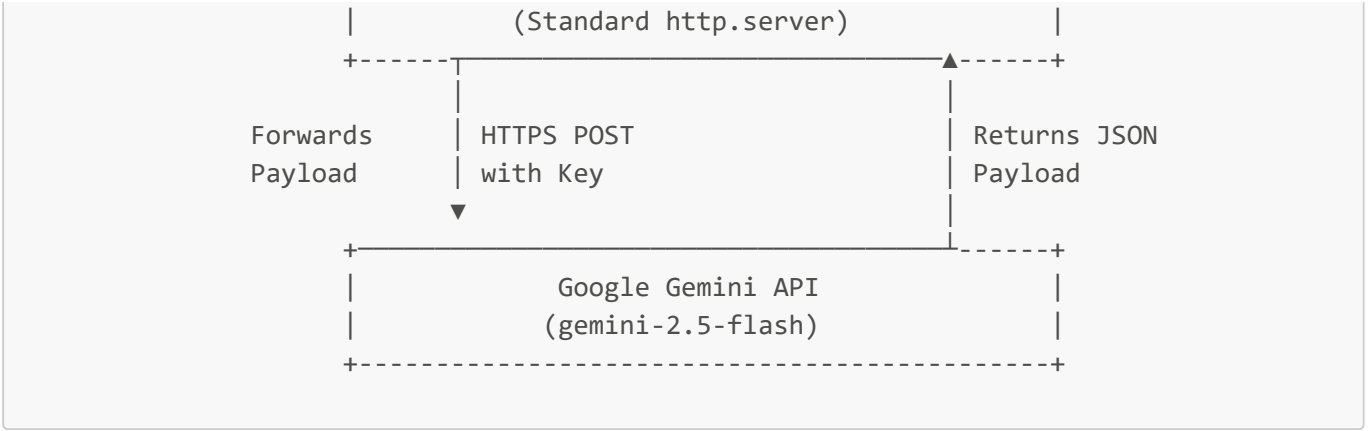
This project, the **AI-Based Resume Analyzer**, is a hybrid web application designed to help job seekers evaluate and optimize their resumes against specific job descriptions. The system leverages natural language processing (NLP) to parse contact information, identify section presence, and analyze skill alignment. When configured with a Google Gemini API Key, the system initiates a deep LLM-based parsing routine to generate personalized feedback, resume bullet point re-writes, and keyword suggestions. If run without API keys, it gracefully defaults to a robust client-side NLP engine, ensuring 100% offline functionality.

2. System Architecture

The application is designed using a lightweight, secure **Client-Server Architecture** optimized for zero local package installation.

Architectural Overview





Key Components

1. **Frontend Client (`Source_Code/public/`)**:
   - `index.html`: Structured using Semantic HTML5 elements. Integrates CDNs for styling (Outfit Fonts, Lucide Icons), plotting (Chart.js), and binary file reading (PDF.js, Mammoth.js).
   - `styles.css`: Features a modern dark theme, glassmorphism card designs, custom scrollbars, and hover transitions. Includes `@media print` rules for clean PDF layout export.
   - `app.js`: Controls browser state, extracts text from files on the user's thread, makes requests to the server, and contains the fallback rules-based parsing engine.
2. **Backend Server (`Source_Code/server.py`)**:
   - Written in pure standard Python. Serves static files, parses `.env` manually, and proxies Gemini API requests. This ensures that the user's private API key remains on the server and is never exposed to the client.
3. **Environment Configuration (`Source_Code/.env`)**:
   - Keeps secret configuration values out of source control.

---

3. Comparative Analysis: AI Tools

Throughout the lifecycle of this project (from initial architecture design to coding, debugging, and drafting documentation), three major AI platforms were evaluated: **ChatGPT**, **Gemini**, and **NotebookLM**. Below is a detailed comparative analysis.

Comparative Analysis Matrix

Parameter	ChatGPT (GPT-4o)	Gemini (Gemini 2.5 Flash / Pro)	NotebookLM
Coding Capability	**Excellent**: Generates optimized code, structures complex layouts, and suggests standard patterns.	**Good**: Fast code generation, occasionally requires prompt refinements for styling details.	**Poor**: Not built for direct code synthesis or formatting.
Debugging Support	**Excellent**: Explains stack traces, detects edge cases, and provides clear diffs.	**Very Good**: Identifies logic errors, but occasionally misses	**Poor**: Cannot compile, test, or analyze code traces interactively.

Parameter	ChatGPT (GPT-4o)	Gemini (Gemini 2.5 Flash / Pro)	NotebookLM
		minute syntax details in large blocks.	
Documentation Quality	**Very Good**: Excellent for creating standard summaries, readmes, and code comments.	**Excellent**: Generates highly detailed technical guides, architecture charts, and reports.	**Excellent**: Synthesizes custom documentation based strictly on uploaded reference documents.
Research Assistance	**Good**: Relies on general training data. May hallucinate details on cutting-edge APIs.	**Very Good**: Integrates live Google Search, providing up-to-date documentation on APIs.	**Outstanding**: Researches and summarizes notes specifically from provided codebase files.
Ease of Use	**Excellent**: Straightforward chat UI, supports multi-modal file uploads.	**Excellent**: Seamless web client and Google integrations.	**Good**: Requires uploading sources first, which can be an extra step.
Best Use Case	Generative coding, script editing, and resolving bug stack traces.	Real-time API research, rapid prototyping, and backend integrations.	Studying existing repositories, reading docs, and generating study notes.

Evaluation Summary

ChatGPT

- **Strengths**: Best-in-class code logic and refactoring. It was highly effective in drafting the CSS grid structure and the Jaccard similarity keyword matching algorithms in JavaScript.
- **Weaknesses**: Knowledge cutoff limits its understanding of newer features in libraries (e.g., PDF.js worker API updates).

Gemini

- **Strengths**: Excels at processing massive contexts and generating JSON schemas directly. It was used to structure the ATS scoring JSON output format and mock payloads, and to write the server.py proxy requests.
- **Weaknesses**: Can write code that is slightly generic, requiring manual adjustment for visual polish.

NotebookLM

- **Strengths**: Outstanding capability to read the entire workspace directory structure, code files, and user instructions, allowing it to generate cohesive documentation and reports without losing context.
- **Weaknesses**: Incapable of writing new code or debugging active runtime errors since it acts primarily as a reading assistant.

---

4. Ethical AI Usage Guidelines

When utilizing AI tools in software engineering, developers must follow these principles:

1. **Verify All Code**: AI models are statistical predictors, not compiler environments. Every generated script, especially complex regex or HTTP request handlers, must be manually tested and verified for syntax and logic.
2. **Avoid Blind Copy-Pasting**: Developers must read and understand the generated code. Blind copying leads to security vulnerabilities, library version mismatches, and bloated codebases.
3. **Mention AI Attribution**: Always mention where AI tools were used (e.g., in `Prompt_Log/` folders). This respects academic integrity and provides transparency on how the system was developed.
4. **Data Security & Privacy**: Never paste active server credentials, personal keys, or private API secrets into public AI chats. Use placeholders or local `.env` variables to isolate keys.
5. **Acknowledge Hallucinations**: Understand that AI can create realistic-looking functions that do not actually exist in the target libraries. Cross-check against official library documentations (e.g., PDF.js or Chart.js reference manuals).

---

5. Manual Improvements

To elevate the AI-generated skeleton, several manual enhancements were applied:

1. **Dynamic Theme Switching UI**: Enhanced the CSS variables to transition smoothly between dark and light themes, ensuring high contrast.
2. **Canvas Reuse Optimization**: Chart.js instances were manually destroyed before redrawing to prevent canvas overlapping bugs during consecutive analyses.
3. **Regex Word-Bounding**: Manually added bounds (`\b`) and special character escaping to ensure technical skills like `C++` and `.NET` match accurately without syntax failures.
4. **Printer Layout styling**: Created dedicated `@media print` rules to enable printing the entire dashboard report beautifully to PDF.